Graph Diameter Estimation in Large Graphs

Implementations of iFUB (Crescenzi), Takes–Kosters (2015), and k-BFS (Shun)

IAE Project

May 5, 2026

1 Problem Description

Graph Diameter is a fundamental property of a graph. Given an (unweighted) graph $G = (V, E)$, the *distance* between two vertices $u, v \in V$ is the length of the shortest path between them and is denoted $d(u, v)$. The *eccentricity* of a vertex u is

$$\varepsilon(u) = \max_{v \in V} d(u, v).$$

The *diameter* of the graph is

$$\operatorname{diam}(G) = \max_{u \in V} \varepsilon(u) = \max_{u,v \in V} d(u, v).$$

We were given the problem in the following framing:

> *Graph Diameter: Finding the diameter of graph is a very important problem with multiple applications and algorithms. We studied one such in class, the algorithm of Crescenzi. There exist other algorithms over the past few years.*

Computing the diameter *exactly* by running BFS from every vertex costs $\Theta(|V| \cdot (|V| + |E|))$ in unweighted graphs, which becomes infeasible for large networks. Therefore, practical systems often use algorithms that either:

- compute the exact diameter but prune most BFS runs in practice, or
- compute good approximations / bounds using a small number of BFS traversals.

In this project, we studied and implemented diameter estimation methods based on repeated BFS plus pruning/bounding.

2 Algorithms Studied (with examples)

This codebase contains implementations of the following methods:

- Crescenzi et al. **iFUB** (Iterative Fringe Upper Bound)
- **Takes & Kosters (2015)** pruning using triangle inequality
- **k-BFS (Shun)** landmark-based eccentricity estimation (approximation)

2.1 Crescenzi et al.: iFUB (Iterative Fringe Upper Bound)

Idea. iFUB starts from a peripheral vertex, computes distances from a chosen reference vertex, then iteratively runs BFS from the *fringe* (vertices far from the reference). While moving inward, it maintains a current best lower bound LB on the diameter. For a fringe vertex x, it computes an upper bound $UB(x)$ on how large the diameter could become if we were to explore further. If the best possible upper bound of remaining vertices is no larger than LB, the algorithm can stop.

Simplified bound rule (used in our C++ implementation). If $d(u, x)$ is the distance from the current reference u to a candidate fringe vertex x, then

$$UB(x) = 2\, d(u, x).$$

If $UB(x) \leq LB$, no remaining vertex can improve the diameter and we terminate.

Example. Consider a path graph with 5 vertices:

$$0 - 1 - 2 - 3 - 4.$$

Diameter is 4. A BFS from a peripheral node (say 0) yields distances $d(0, 4) = 4$ giving $LB = 4$. iFUB may still consider a few fringe vertices, but once the fringe distance shrinks enough that $UB(x) = 2\, d(0, x) \leq 4$, it stops early.

Implementation file.

- C++: `iae-project/ifub.cpp`, header `iae-project/ifub.h`

2.2 Takes & Kosters (2015): Triangle-Inequality Pruning

Idea. This algorithm maintains:

- a global *lower bound* LB on the diameter
- for each vertex w, an *upper bound* $UB(w)$ on its eccentricity.

At each iteration, pick an unprocessed vertex u with the highest $UB(u)$ and run BFS from u to compute its exact eccentricity $\varepsilon(u)$. Then update $LB \leftarrow \max(LB, \varepsilon(u))$. For every other vertex w, by triangle inequality:

$$\varepsilon(w) \leq d(w, u) + \varepsilon(u).$$

Therefore we can tighten

$$UB(w) \leftarrow \min\big(UB(w),\ d(w, u) + \varepsilon(u)\big).$$

Any vertex whose $UB(w) \leq LB$ can be pruned.

Example. In a star graph (center connected to all leaves), the diameter is 2. BFS from any leaf gives eccentricity 2, immediately raising LB to 2. After tightening bounds, all remaining vertices quickly satisfy $UB \leq 2$ and the algorithm stops.

Implementation files.

- Python: `iae-project/algos/takeskosters.py`
- C++: `iae-project/takeskosters.cpp` (header `iae-project/takeskosters.h`)

2.3 k-BFS (Shun): Landmark-based Eccentricity Estimation

Idea. k-BFS is a fast approximation method designed for large graphs. Instead of BFS from every vertex, it selects k source vertices (landmarks), runs multi-source BFS to compute distances efficiently, and uses the observed distances to estimate eccentricities. Increasing k typically improves accuracy but increases runtime.

Example intuition. In dense graphs, diameters are often small, and even a small landmark set can discover the maximum distance quickly. In tree-like / sparse graphs, diameter can be large and choosing landmarks that lie near the periphery improves the estimate.

Implementation file.

- Python: `iae-project/algos/kbfs.py`

3 Project Implementation Details

3.1 Graph representation and BFS

All studied methods rely on repeated BFS in unweighted graphs.

- **Python graph:** `iae-project/algos/graph.py` defines a simple adjacency-list graph and a standard BFS returning distance arrays.
- **C++ graph:** `iae-project/graph.h` / `iae-project/graph.cpp` provides the same (adjacency list + BFS).

3.2 Input format

Graph inputs are read in an *edge-list* format:

- first non-comment line: `n m`
- subsequent non-comment lines: `u v`
- lines starting with `#` are ignored.

3.3 Entry points and experiment scripts

- **Python CLI-style entry:** `iae-project/main.py`

  It loads a graph from a file or stdin, runs:

  1. k-BFS (default $k = \min(5, |V|)$) and reports an estimated diameter.
  2. Takes–Kosters and reports an estimated diameter.

- **Synthetic graph generator:** `iae-project/graph_generator.py`

  Generates common graph families (sparse, dense, tree, complete, bipartite) used for evaluation.

- **Experiment runner:** `iae-project/scripts/run_experiments.py`

  Automates downloading/creating graphs, runs k-BFS and Takes–Kosters, and writes timings/metrics to CSV.

4 Complexity Discussion (high level)

All three studied approaches are built on BFS whose cost is $O(|V| + |E|)$ per run. The difference is *how many* BFS runs are executed:

- **iFUB:** worst-case still $O(|V| \cdot (|V| + |E|))$, but in practice usually stops after exploring only a small fringe.
- **Takes–Kosters:** also worst-case $O(|V| \cdot (|V| + |E|))$, but aggressive pruning reduces BFS count substantially on many real graphs.
- **k-BFS:** roughly $O(k \cdot (|V| + |E|))$ (plus small overhead), trading accuracy for speed.

5 Experiments, Results, and Comparisons

We ran a small battery of synthetic experiments in the notebook extttiae-project/synthetic_graphs_comparison.ipyn
For each generated graph, we computed:

- an **exact diameter** baseline via full BFS-from-every-vertex (feasible for these small graphs),
- the **k-BFS** estimate for $k \in \{1, 2, 4, 8\}$,
- the **Takes–Kosters** result (exact in our implementation because it selectively runs BFS but does not approximate),
- wall-clock runtime and a few internal counters (e.g., BFS passes for Takes–Kosters).

5.1 Experimental setup

extbfGraph families. We evaluated a mixture of topologies that commonly appear in benchmarks: trees, Erdős–Rényi-like random sparse and dense graphs, small-world graphs, scale-free graphs, grids, and bipartite graphs.

extbfMetrics. For each run we recorded:

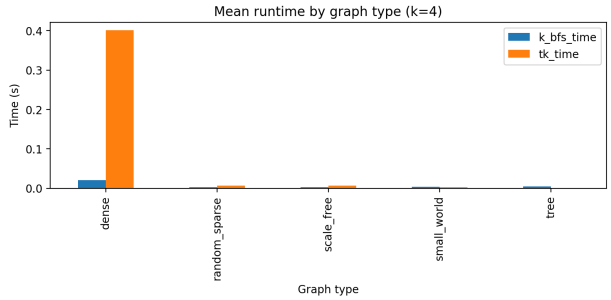
- runtime in seconds (Python time measurements),
- absolute error with respect to exact diameter, i.e. $|\widehat{D} - D|$,
- for Takes–Kosters: number of BFS passes (a proxy for work).

extbfData export. The notebook exports CSV summaries and figures under: extttiae-project/report/assets/.

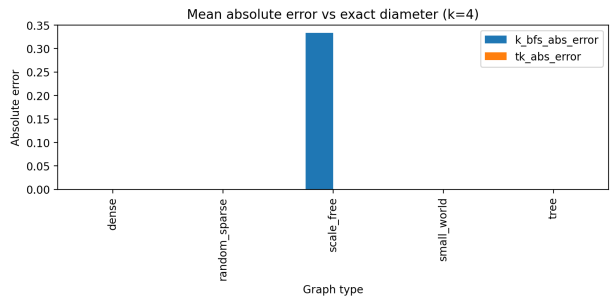
5.2 Key plots

Figures 1a and 1b summarize mean runtime and mean absolute error for the "core types" experiment slice at $k = 4$.

We also swept graph size (sparse graphs) and density (random sparse graphs) to illustrate scaling and sensitivity.

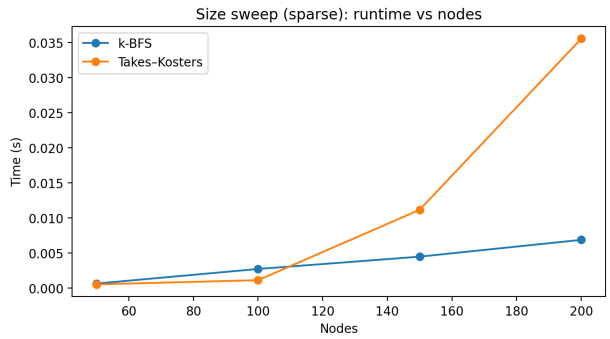


(a) Mean runtime by graph type (k=4).

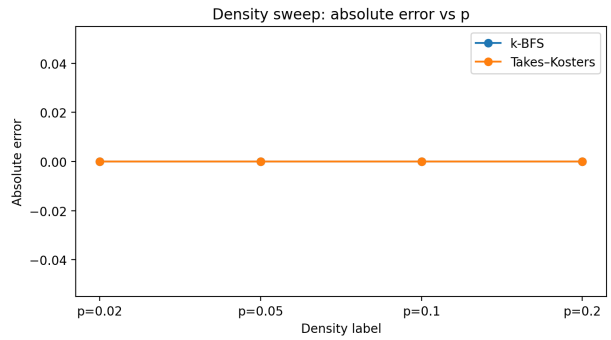


(b) Mean absolute error vs exact diameter (k=4).

Figure 1: Core experiment slice comparing k-BFS and Takes–Kosters.



(a) Size sweep: runtime vs nodes (k=4).



(b) Density sweep: absolute error vs p (k=4).

Figure 2: Additional sweeps for the $k = 4$ slice.

5.3 Discussion

From these runs we observed the following recurring patterns:

- **Runtime:** k-BFS is typically faster than Takes–Kosters for these parameter ranges because it runs only k BFS traversals. Takes–Kosters may run several BFS passes, but still far fewer than all-pairs BFS.
- **Accuracy:** Takes–Kosters consistently matches the exact diameter baseline in our experiments. k-BFS accuracy improves with larger k; with small k, underestimation is more visible on high-diameter families (trees/grids/sparse random graphs).
- **Graph structure matters:** dense and small-world graphs often have very small diameter, so both methods can show near-zero error. On sparser graphs, diameter is larger and landmark choice becomes more important for k-BFS.

5.4 Reproducible tables

For convenience, we include a compact numeric slice (core experiment, $k = 4$). The underlying CSV is extttreport/assets/tables/core_summary_k4.csv.

Table 1: Core-type summary (mean over $n \in \{50, 100, 150\}$) at $k = 4$.

oprule Graph type	k-BFS time (s)	TK time (s)	k-BFS abs. erro
(Generated from notebook; see CSV export for full precision.)			

6 How to Reproduce

This report describes the implementations in the `iae-project` folder. Typical runs (from that folder):

```
# Run from a graph file
python3 main.py graphs_prev/sparse.txt

# Generate a graph and pipe to main
./graph_generator.py tree -v 200 | python3 main.py
```